FH Salzburg
MultiMediaTechnology

Basics Game Development

Asset Handling, Rendering Pipeline

Technology
Health
Media

Asset Handling

What are Game-Assets?

Game assets include graphics (characters, environments, logos), background music, special effects, and sound effects. There are many tools and resources available to create your own game assets or use assets that already exist.

©

Always make sure you own the rights to an asset before using it in your game.

Audio

It is the music that pulls you into the world that the game is trying to show you. Essentially, it is the sound that really creates a large amount of the emotion that one feels when playing a game.

- **Background Music / Soundtracks:**
  - Use compressed formats like **mp3** or **ogg**. This will keep your memory footprint low but costs you some cpu-cycles for decoding. (Music class in SFML)
- **Short Clips / Sound Effects (SFX):**
  - Use uncompressed formats like **wav**. This will take up more space in your memory but is more efficient in the end because of the decoder overhead. (Sound class in SFML)

Audio Implementation

- **2D Audio:**
  - The Audio is played directly to your Speakers (inside your virtual ears). You can only control the volume (loudness).
  - Usually used for background music, instructions and UI.
- **3D Audio:**
  - Spatial position/movement in the game-world.
  - Physical effects like Doppler-Effect, reverb zones etc.
  - Some libraries can even simulate physical properties of materials and deal with occlusions.
  - 3D audio is often used in 2D games to.
  - Available in SFML
- Current industry leading game-audio-solutions: “FMOD-Studio” and “Wwise”.

There is usually an upper limit, depending on platform and environment, how many sounds you can play simultaneously. SFML supports 256 simultaneous sounds.

Audio - Online Sources

- https://sonic-pi.net/ **code** your own music 💪
- https://freesound.org/
- https://opengameart.org/
- https://pixabay.com/
- https://www.zapsplat.com/
- AIVA - The AI composing emotional soundtrack music
- Royalty Free Music by Beatoven | Beatoven.ai
- Elevenlabs
- Suno.ai

- MMA-Audio Students 🏆

Images

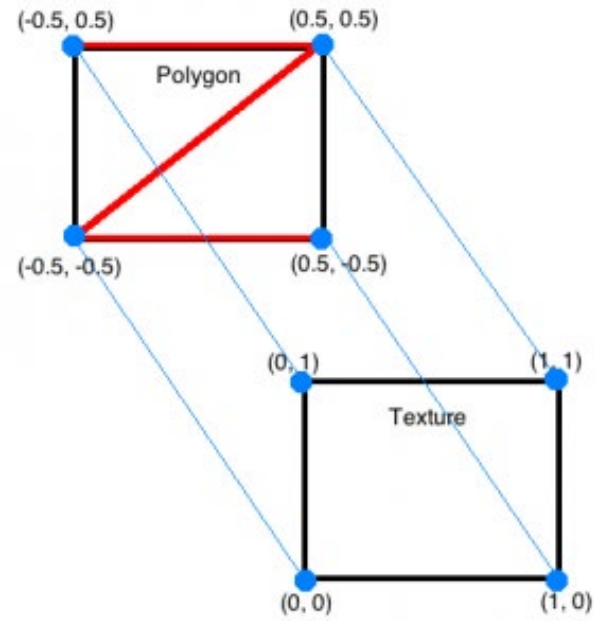
Image-files are a cruical data-source in every game. They define the look of materials and the look of the game in general. In addition, there are many other areas of application for image files. For example, for light calculations or as occlusion masks.

- **Raster Images:**
  - 2D-Array of color-values.
  - Calculation effort increases quadratically with the image size.
  - File formats: **PNG**, JPEG, TIFF, webp, …
- **Vector Images:**
  - Consist of code (Points, Arcs, Lines, Colors, …)
  - Calculation effort increases linearly with the number of instructions.
  - File formats: SVG, EPS, PDF, AI, …
  - Vector Images must be precomputed (Rasterized) on the CPU.

Rule of Thumb:
To maximize hardware compatibility, keep your **image-size** always at a **power of 2**.

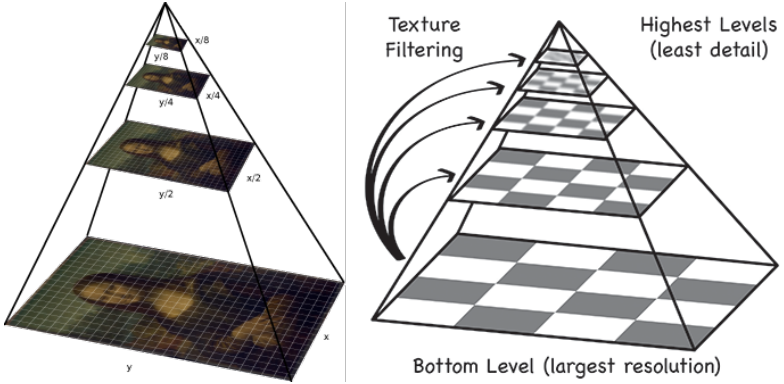
Images – How is a texture loaded?

- Texture image originates as an array of **texels** in CPU memory.
- **A texel is a texture element**: i.e. the **color** to be applied to an **interpolated fragment.**



Images – Mipmaps

- In computer graphics, mipmaps (also MIP maps) or pyramids are pre-calculated, optimized sequences of images, each of which is a progressively lower resolution representation of the previous.
- In 3D games mipmaps levels are used to display a lower resolution texture on objects that are far away.
- In 2D Games Mipmaps are useful when textures are scaled down. You can activate mipmaps in SFML on texture-level.
- Usually you can select the interpolation-method (Texture Filtering) that is applied between the mipmap levels (bilinear / trilinear).



Images – Smooth Filter in SFML

- The SFML documentation does not give us any information about how the smooth filter works for the texture. However, since SFML is an open source project...

```
glCheck(glBindTexture(GL_TEXTURE_2D, m_texture));
glCheck(glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, m_isSmooth ? GL_LINEAR : GL_NEAREST));

if (m_hasMipmap)
{
    glCheck(glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, m_isSmooth ? GL_LINEAR_MIPMAP_LINEAR : GL_NEAREST_MIPMAP_LINEAR));
}
else
{
    glCheck(glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, m_isSmooth ? GL_LINEAR : GL_NEAREST));
}
```

GL_NEAREST

GL_LINEAR

Images – Smooth Filter in SFML (Details)

https://www.khronos.org/registry/OpenGL-Refpages/es2.0/xhtml/glTexParameter.xml

- **GL_TEXTURE_MIN_FILTER:**The texture **minifying function** is used whenever the **pixel** being textured maps to an **area greater than one texture element**. There are six defined minifying functions. Two of them use the nearest one or nearest four texture elements to compute the texture value. The other four use mipmaps. Here are the four used by SFML:
  - **GL_NEAREST**: Returns the value of the **texture element that is nearest** (in Manhattan distance) to the center of the pixel being textured.
  - **GL_LINEAR**: Returns the **weighted average of the four texture elements that are closest to the center** of the pixel being textured.
  - **GL_NEAREST_MIPMAP_LINEAR**: Chooses the **two mipmaps that most closely match the size of the pixel** being textured **and uses the GL_NEAREST** criterion (the texture element nearest to the center of the pixel) to produce a texture value from each mipmap. The final texture value is a **weighted average of those two values**.
  - **GL_LINEAR_MIPMAP_LINEAR**: Chooses **the two mipmaps that most closely match the size of the pixel** being textured and uses the **GL_LINEAR** criterion (a weighted average of the four texture elements that are closest to the center of the pixel) to produce a texture value from each mipmap. The final texture value is a **weighted average of those two values**.

Images – Smooth Filter in SFML (Details)

https://www.khronos.org/registry/OpenGL-Refpages/es2.0/xhtml/glTexParameter.xml

- **GL_TEXTURE_MAG_FILTER:** The texture **magnification function** is used when the **pixel** being textured maps to an **area less than or equal** to one **texture element**. It sets the texture magnification function to either GL_NEAREST or GL_LINEAR (see below). GL_NEAREST is generally faster than GL_LINEAR, but it can produce textured images with sharper edges because the transition between texture elements is not as smooth. The initial value of GL_TEXTURE_MAG_FILTER is GL_LINEAR.
  - **GL_NEAREST:** Returns the value of the **texture element that is nearest** (in Manhattan distance) to the center of the pixel being textured.
  - **GL_LINEAR:** Returns **the weighted average of the four texture elements that are closest** to the center of the pixel being textured.

Images – Online Sources

- https://devassets.com/
- https://substance3d.adobe.com/community-assets
- https://www.gameart2d.com/freebies.html
- https://opengameart.org/content/2d-complete-characters
- https://www.kenney.nl/assets
- https://itch.io/game-assets/free

- MMA Communication-Design Students 🏆
- MMA 3D-Art Students 🏆

Models (3D)

- A jungle of file formats created by a multitude of different 3D programs.
- Here are two widely used exchange-formats:
  - **OBJ**:
    - Very old but simple file-format.
    - Human readable ASCII text.
    - Only Vertices and Materials can be stored.
  - **FBX**:
    - Invented by Autodesk to increase the compatibility between their programs (AutoCAD, Maya, 3ds Max, …)
    - Can also store vertices and materials but additionally also complete object-hierarchies, animations, lighting, cameras, animation-bones, blend-shapes etc.

Models – Online Sources

- https://www.turbosquid.com/
- https://www.mixamo.com/#/
- https://assetstore.unity.com/

- MMA 3D-Art Students 🏆

Other Asset-Types

- **Structured Text Files**
  - Scene / Level Structure
  - Shader
  - Scripts
  - Animation clips
  - Settings
  - …
- **Videos** (**Tipp:** Try to choose an encryption format that is natively decryptable on the target platform. MP4 h264 is in most cases a good bet.)
- **Plugins / Libraries**
- **Concept Art**
- …

Runtime Memory Management

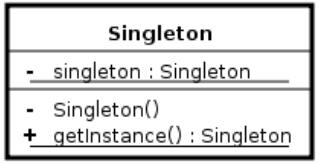
C# provides automatic memory management. This is sometimes annoying in game development. Especially garbage collection can cost you a bunch of time. To prevent unexpected frame-drops during gameplay, we must **manage** our **memory in an indirect way**…

- **General goals**:
  - **Avoid heap memory allocations** inside game loop.
  - **Avoid garbage collection** inside game loop.

- **Strategies to meet this goals:**
  - **Load all necessary assets into memory before the game loop starts (Asset Manager).**
  - **Instantiate all required objects before the game-loop starts** and hold the references until the game loop ends or pauses (GameObject Manager).
  - For further optimization, **use "Object-Pooling"** for objects with a short lifetime (like bullets) (Instantiate a pool of objects but let them deactivated until you need them. Then deactivate them again and reuse them later instead of destroying and re-instantiating it).

Asset Manager

- **The Problem:** Loading Assets at different places in your program can lead to a very hard maintainable code structure.
- **The Idea:** Let's load/unload all assets at **a central place in our program** and make them easily accessible within our whole codebase (through simple asset names or id’s).
- **The Solution:** A globally accessible Manager-Class that loads all required assets and holds the references **in dictionaries** (aka. hash-map data structure).



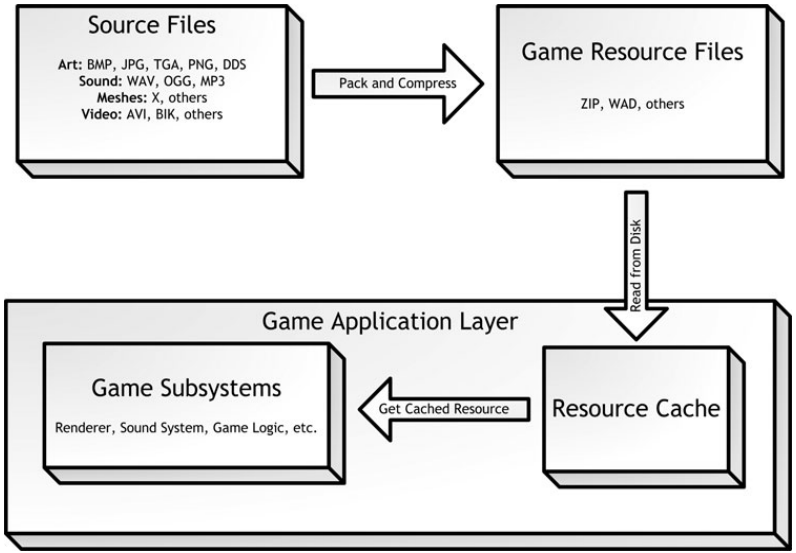
Such manager classes are usually implemented following the **singleton design pattern** (singleton is a **common but controverse** design-pattern, we will learn more about singletons in the lab).

Challenges

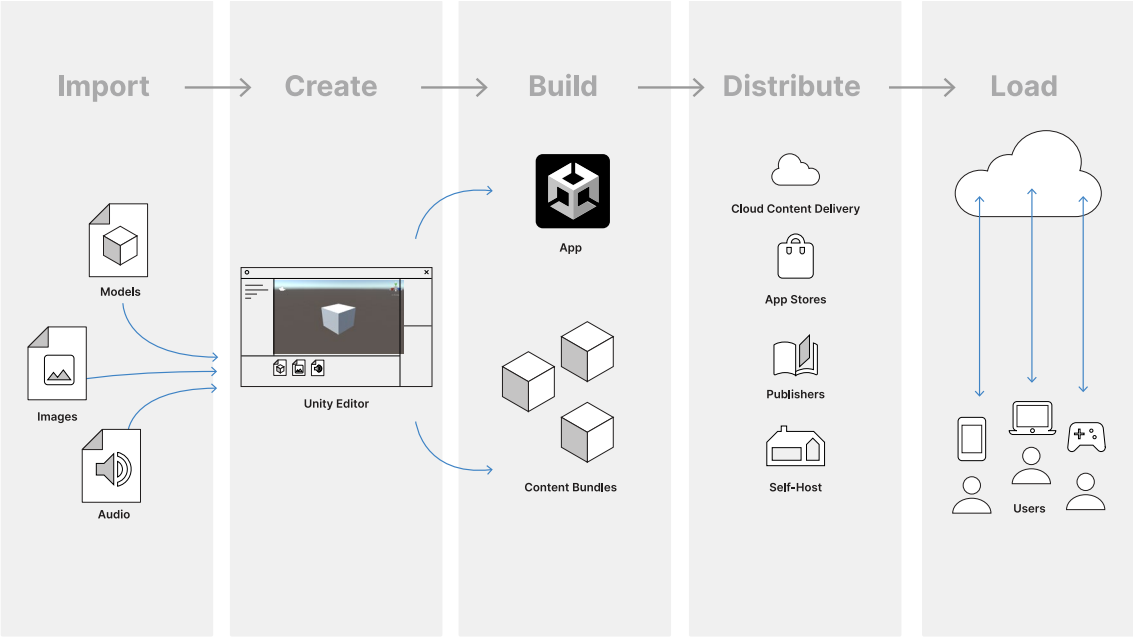
- **Production-Pipeline stability:** During the development of a game, especially when following an iterative workflow, you must deal with frequent asset changes. Try to setup a pipeline workflow that avoids manual work when it comes to asset changes.
- **Multi-platform compatibility:** Some target platforms may require special file formats or encodings to work properly. Multiplatform development environments must deal with this problem. Unity for example is internally converting these formats automatically for you. SFML supports also multiple platforms but has no built-in converters.
- **Updates**: Asset management is also an important part when it comes to content updates after release. The update-strategy for assets must be clarified and planed in advance.

Asset Packages

- A solution to some of the challenges discussed on the previous slide is to extract your game-assets in more abstracted **game resource files**. (See Game Coding Complete Chapter 8)



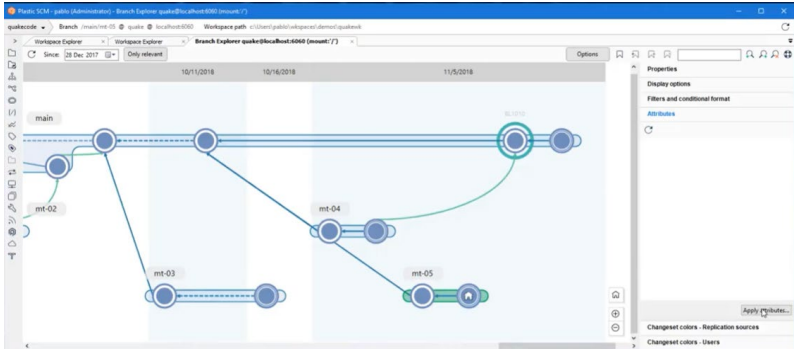
Unity Asset Workflow



Game Assets and Version Control Systems (VCS)

Game Assets are often BLOBs (Binary Large Objects).
You should normally try **to avoid BLOB‘s in your VCS**.

- **Optimization Strategies:**
  - **Prefer ASCII** based file formats (Text Files).
  - If you use GIT, then **setup GIT-LFS** (Large File System) to keep your local repository small and fast (requirement 3. semester game). This also helps us managing the storage of our GitLab-server.
  - **Ignore junk**-files and -folders.
  - Introduce a **naming convention** for assets and code.
  - Use a **specialized VCS** like “Plastic SCM” or “Perforce”. They are highly optimized in terms of dealing with different kind of assets and code and have also a lot of “artist friendly” features built in.



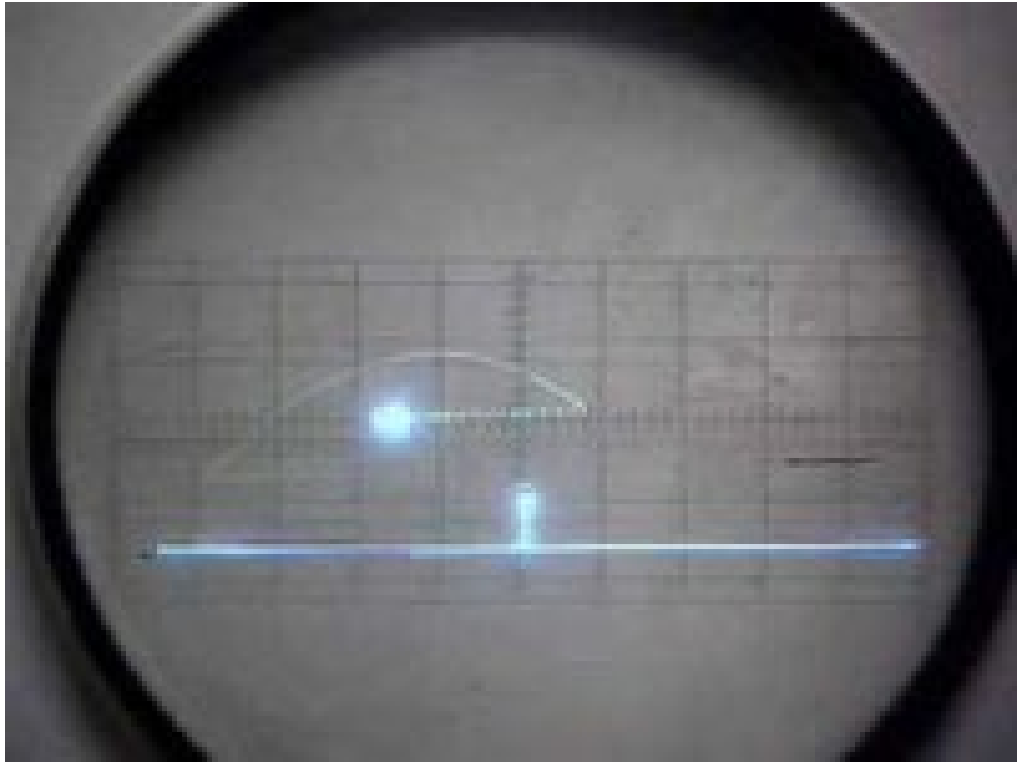
Render Pipeline

What is Rendering?

In graphics the word 'render' means the process of converting information about geometric objects into a visual image on your computer screen. **Rendering** a scene **is a multi-step process** which is often described in terms of a **graphics pipeline**.

At one end of the pipeline **we feed in polygon vertices**, color information about the objects in our scene, bitmaps to paint onto some of the scene objects or use as backgrounds, and perhaps some locations of lights. At the other end of the pipeline a **rendered two-dimensional color image** appears in a memory location called the **frame buffer**.

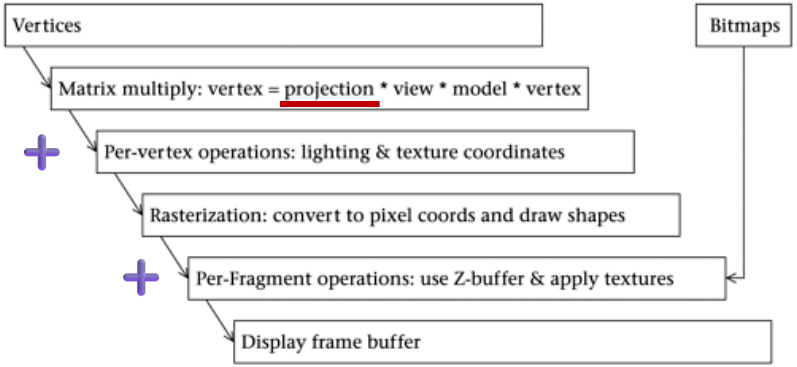
Real Time Rendering – First Steps (1958)



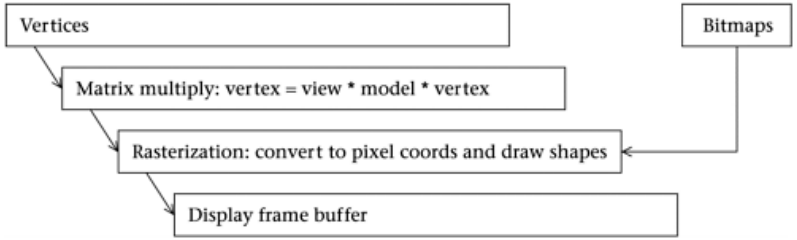
Real Time Rendering – State of the Art (UE5)

“Difference” between 2D and 3D Render Pipeline

Simplified 3D Render Pipeline



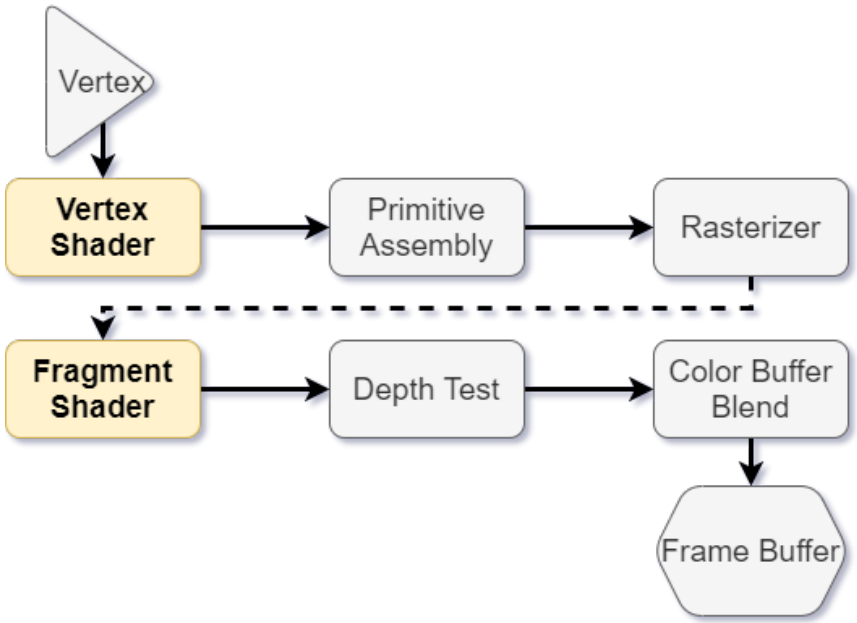
Simplified 2D Render Pipeline



Some steps, like projection, could be theoretically skipped or optimized for a 2D render pipeline. However, since most engines also use the entire graphics pipeline for 2D games, we will look at the programmable 3D pipeline in detail.

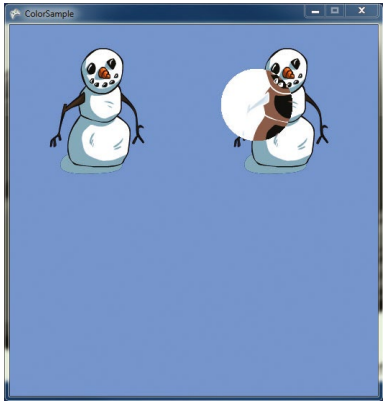
Images taken from etutorials

Simplified Programmable Render Pipeline



Why Programmable Pipeline?

- So far, we have manipulated the scale, orientation, and color of sprites to create various effects. But what happens if we want to modify individual pixels, as required to create a blur effect?
- Even if we know how to modify a single texture, we can take this a step further. Let's assume we have created a complicated scene involving multiple sprites at various layers. Suppose also that we now want to apply an effect (like the inverted circle in the snowman image) across the entire scene.
- The calculation effort needed to archive this is very high (to high for CPU).
- This is the point where the **Programmable Render Pipeline** joins the game.
- We can write small programs (called “**Shaders**”) that are directly executed on the Graphics Card.



Shaders

- In computer graphics, a shader is a type of computer program originally used for shading in 3D scenes (the production of appropriate levels of light, darkness, and color in a rendered image). They now perform a variety of specialized functions in various fields within the category of computer graphics special effects, or else do video post-processing unrelated to shading, or even perform functions unrelated to graphics (compute shaders).
- **Examples:**
  - Light and Shadows
  - Animations
  - Particles
  - Materials
  - Ray Tracing
  - Fluids …

Drone Swarm, Stillalive Studios: Drones are not only rendered by shaders but also controlled by shaders.

Shading Languages (Real Time)

- **OpenGL (GLSL):** Very popular (industry standard). GLSL works together with SFML.
- **Cg:** Developed by NVIDIA, was designed for easy and efficient production pipeline integration. Development of Cg was stopped in 2012 and the language is now deprecated.
- **DirectX:** The High-Level Shading Language (also called HLSL for short) is a C-style shader language for DirectX 9 and higher and Xbox game consoles. It is related to Nvidia's Cg but is only supported by DirectX and Xbox. HLSL programs are compiled into bytecode equivalent of DirectX shader assembly language.
- **Metal:** Apple has created a low-level graphics API called Metal, which runs on most Macs made since 2012, iPhones since the 5S, and iPads since the iPad Air. Metal introduces its own shading language called Metal Shading Language (MSL).

Vertex Shader Example (Wave)

```
uniform float wave_phase;
uniform vec2 wave_amplitude;

void main()
{
    vec4 vertex = gl_Vertex;
    vertex.x += cos(gl_Vertex.y * 0.02 + wave_phase * 3.8) * wave_amplitude.x
                + sin(gl_Vertex.y * 0.02 + wave_phase * 6.3) * wave_amplitude.x * 0.3;
    vertex.y += sin(gl_Vertex.x * 0.02 + wave_phase * 2.4) * wave_amplitude.y
                + cos(gl_Vertex.x * 0.02 + wave_phase * 5.2) * wave_amplitude.y * 0.3;

    gl_Position = gl_ModelViewProjectionMatrix * vertex;
    gl_TexCoord[0] = gl_TextureMatrix[0] * gl_MultiTexCoord0;
    gl_FrontColor = gl_Color;
}
```

Example in SFML.
`Wave_phase` and `wave_amplitude` are calculated on CPU based on Mouse coordinates.

Vertex Shader Example (Wave)

```
uniform float wave_phase;
uniform vec2 wave_amplitude;

void main()
{
    vec4 vertex = gl_Ve
    vertex.x += cos(gl_Ver
                + sin(gl_Vertex
    vertex.y += sin(gl_Ve
                + cos(
                                                x
                                                0.3;
                                                tude.y * 0.3;

    gl_Position = gl_ModelViewProj
    gl_TexCoord[0] = gl_TextureMa x[0] *  ciTex
    gl_FrontColor = gl_Color;
}
```

Parallel execution on GPU for every <u>VERTEX</u> of the <u>geometry</u> 🤩

Example in SFML.
`Wave_phase` and `wave_amplitude` are calculated on CPU based on Mouse coordinates.

Fragment Shader Example (Blur)

```
uniform sampler2D texture;
uniform float blur_radius;

void main()
{
    vec2 offx = vec2(blur_radius, 0.0);
    vec2 offy = vec2(0.0, blur_radius);

    vec4 pixel = texture2D(texture, gl_TexCoord[0].xy)              * 4.0 +
                 texture2D(texture, gl_TexCoord[0].xy - offx)       * 2.0 +
                 texture2D(texture, gl_TexCoord[0].xy + offx)       * 2.0 +
                 texture2D(texture, gl_TexCoord[0].xy - offy)       * 2.0 +
                 texture2D(texture, gl_TexCoord[0].xy + offy)       * 2.0 +
                 texture2D(texture, gl_TexCoord[0].xy - offx - offy) * 1.0 +
                 texture2D(texture, gl_TexCoord[0].xy - offx + offy) * 1.0 +
                 texture2D(texture, gl_TexCoord[0].xy + offx - offy) * 1.0 +
                 texture2D(texture, gl_TexCoord[0].xy + offx + offy) * 1.0;

    gl_FragColor =  gl_Color * (pixel / 16.0);
}
```

Example in SFML.
`Blur_radius` and `texture` are set from CPU side.

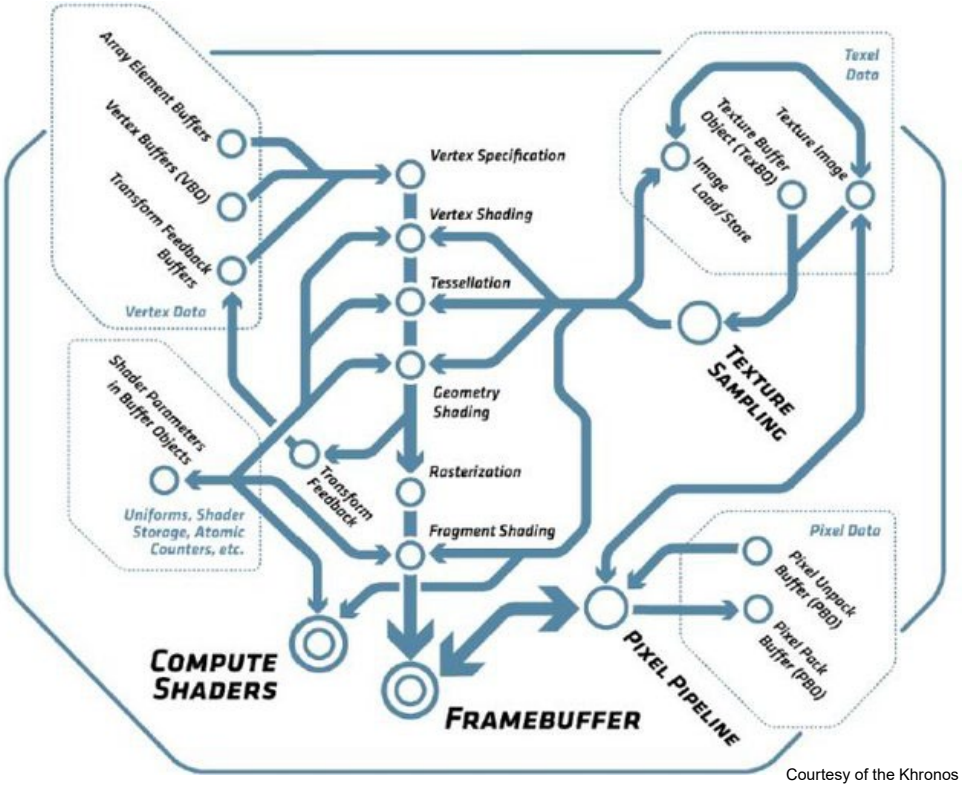
Fragment Shader Example (Blur)

```
uniform sampler2D texture;
uniform float blur_radius;

void main()
{
    vec2 offx = vec2(blur_
    vec2 offy = vec2(0.0, blu

    vec4 pixel = texture2D(te
                 tex
                 texture
                 texture2D(
                 texture2D(te
                 texture2
                 text
                 texture2D(texture,
                 texture2D(texture,
                                    xy
                                    offy,

    gl_FragColor =  gl_Color * (pixel / 16.0
}
```

Parallel execution on GPU for every <u>fragment</u> (pixel) of the final <u>image</u> 🤩

Example in SFML.
`Blur_radius` and `texture` are set from CPU side.

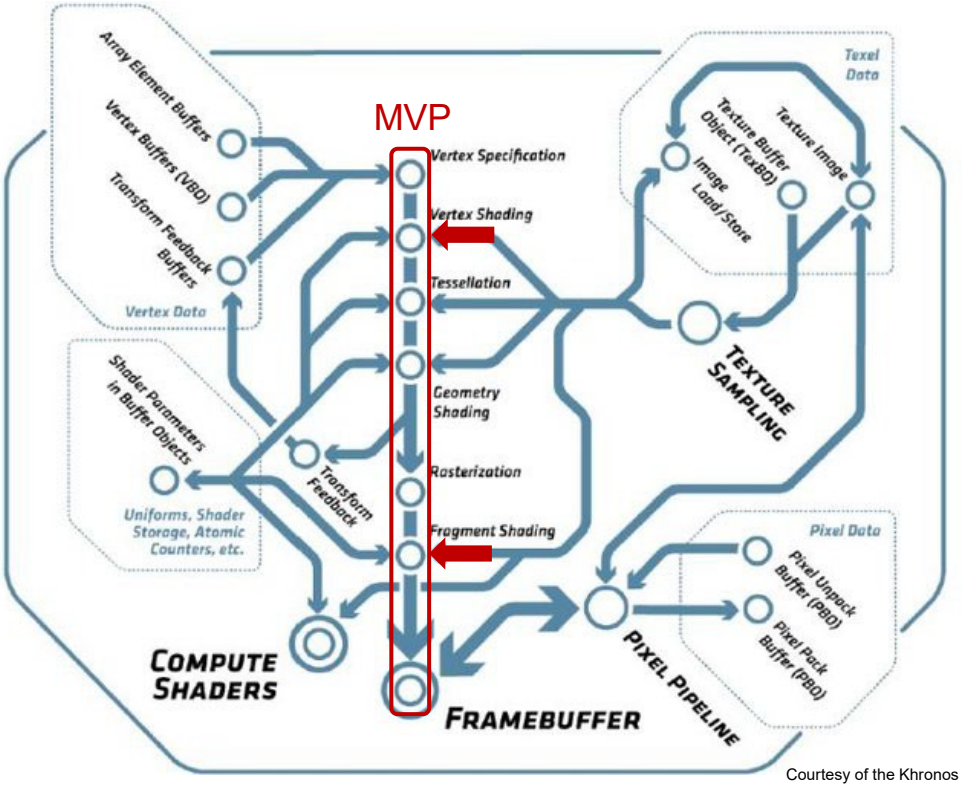
OpenGL 4.0 Core Profile



Courtesy of the Khronos Group

Why always simplified versions of the graphics pipeline? 🤔

OpenGL 4.0 Core Profile



Courtesy of the Khronos Group

Pixelate Shader

Let’s analyze a shorter fragment shader example (without vertex shader) …

```
uniform sampler2D texture;
uniform float pixel_threshold;

void main()
{
    float factor = 1.0 / (pixel_threshold + 0.001);
    vec2 pos = floor(gl_TexCoord[0].xy * factor + 0.5) / factor;
    gl_FragColor = texture2D(texture, pos) * gl_Color;
}
```

Fragment Shader Example in SFML.
`pixel_threshold` is calculated on CPU based on Mouse coordinates.

Pixelate Shader

„Uniform“ data we sent from the CPU to the GPU.

```
uniform sampler2D texture;
uniform float pixel_threshold;

void main()
{
    float factor = 1.0 / (pixel_threshold + 0.001);
    vec2 pos = floor(gl_TexCoord[0].xy * factor + 0.5) / factor;
    gl_FragColor = texture2D(texture, pos) * gl_Color;
}
```

current texture (UV) coordinates

Pixelate Shader

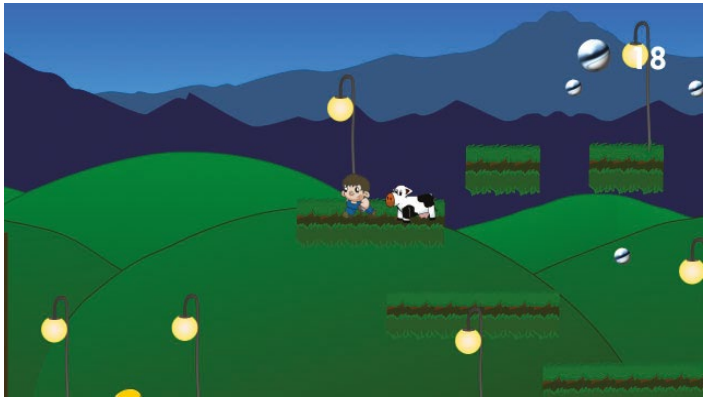
```
uniform sampler2D texture;
uniform float pixel_threshold;

void main()
{
    float factor = 1.0 / (pixel_threshold + 0.001);
    vec2 pos = floor(gl_TexCoord[0].xy * factor + 0.5) / factor;
    gl_FragColor = texture2D(texture, pos) * gl_Color;
}
```

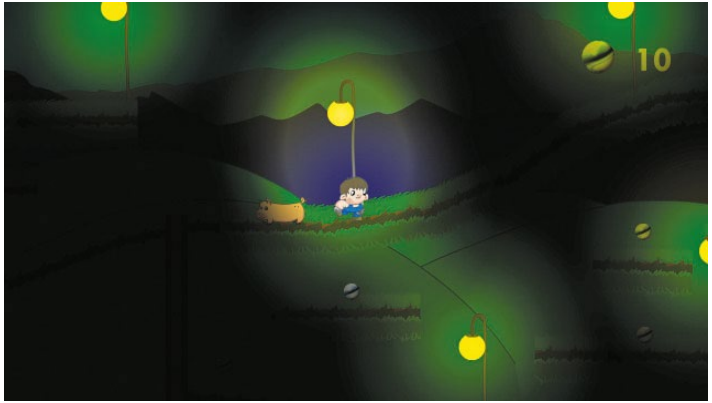
Color set by SFML

Resulting color for the fragment.

Sampling color from texture.

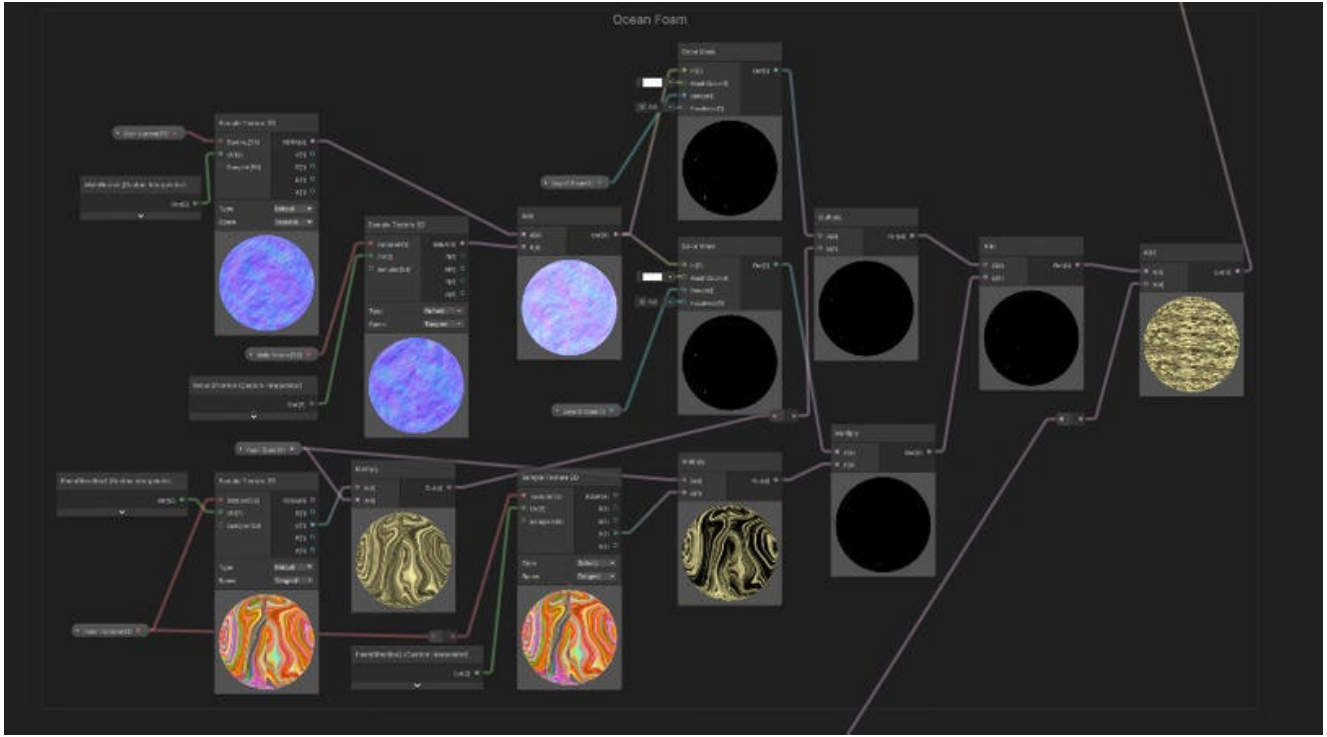
2D Light Example





Advanced light shader with multiple lights sources, before (left) and after (right), byThomas Francis.

Visual Scripting Shader-Editors



Unity Shader Graph

Visual Scripting Shader-Editors

- **Pro:**
  - Intuitive workflow if you already used to material editors in 3D-Modeling Software.
  - Realtime feedback.
  - No coding errors.
- **Con:**
  - Bad compatibility with traditional shaders.
  - Often hard to understand for programmers (abstracted workflow).
  - Limited functionality.
  - Overhead.
  - Hard to read and modify.
  - Easy to produce underperforming shaders.

Online Ressources

- **Playgrounds:**
  - https://www.shadertoy.com/
  - https://glslsandbox.com/
- **WebGL Examples:**
  - https://www.sliderrevolution.com/templates/portal-effect-hero-slider/?utm_medium=inline-ad&utm_source=webgl-examples
  - https://www.sliderrevolution.com/templates/cyber-particle-effect/?utm_medium=inline-ad&utm_source=webgl-examples
  - https://webglsamples.org/collectibles/index.html

CG skills are also useful for web developers!

FH Salzburg
MultiMediaTechnology

Basics Game Development

Asset Handling, Rendering - End

Technology
Health
Media

© 2025 Fachhochschule Salzburg GmbH
www.fh-salzburg.ac.at